

1교시: Runtime config와 env 주입

Day 4: Runtime Config & Observability

Docker 런타임 검증 지도: 문제를 빠르게 발견하고, 원인 파악 후, 안전하게 정리하자!

목표
안정적인 운영과 빠른 문제 해결

자주 발생하는 실패 사례

- 환경변수 누락**
 - 필수 환경변수 미설정
 - .env 파일 경로/형식 오류
- 잘못된 포트**
 - 애플리케이션 포트 불일치
 - 호스트 포트 매핑 오류
- 잘못된 네트워크**
 - 컨테이너가 다른 네트워크
 - 서비스 간 통신 실패
- 스테일 볼륨**
 - 오래된 데이터 잔존
 - 볼륨 마운트 경로 오류
- 잘못된 이미지 태그**
 - 존재하지 않는 태그
 - 잘못된 버전 번호

1 Runtime Config & Secret 비노출	2 docker logs & Readiness	3 docker inspect 메타데이터 확인	4 docker exec 컨테이너 내부 진입	5 docker stats & Restart Policy	6 Failure Drill 문제 재현 & 분석	7 Cleanup Audit 정리 & 감사
<code>.env / env-file</code> <code>Secret 마스킹</code> <code>docker run --env-file .env ...</code> <code>docker compose --env-file .env up</code> 검증 포인트 - 필수 환경변수 존재 확인 - 기존값/포트/타입 검증 - Secret 값은 마스킹 처리 (로그, inspect 출력 주의)	<code>docker logs -f <container></code> <code>docker logs --tail 100 <container></code> 검증 포인트 - 여러/스케터레이스 확인 - 애플리케이션 시작 로그 - Readiness/Health 상태 확인 (HEALTHCHECK 활용)	<code>docker inspect <container></code> <code>docker inspect <image></code> 검증 포인트 - Env / Port / Volume 확인 - Network / IP / DNS 확인 - Restart Policy / Health / Mount / Command 확인	<code>docker exec -it <container> sh</code> <code>docker exec -it <container> bash</code> 검증 포인트 - 파일/설정/변수 확인 - 네트워크 연결 테스트 (curl, ping, nslookup 등) - 프로세스/포트 상태 확인 (ps, ss/netstat 등)	<code>docker stats <container></code> <code>docker inspect <container> (RestartPolicy)</code> 검증 포인트 - CPU / MEM / NET / I/O 확인 - Restart Policy 동작 확인 (no / on-failure / always) - OOM / 재시작 횟수 확인	문제 상황을 재현하여 로그/메트릭/설정 기반으로 원인 분석 검증 포인트 - 가설 수립 - 관찰 데이터 수집 - 변형 시범 기록 - 해결/유지 방안 도출	<code>docker system prune -af</code> <code>docker image prune -af</code> 검증 포인트 - 불필요한 컨테이너/이미지/네트워크/볼륨 정리 - 변형 사항 기록 - 비용/보안/정합성 점검

핵심 원칙
관찰 → 진단 → 원인 파악 → 해결 → 검증 → 정리
반복 가능한 프로세스로 운영 안정성 확보!

운영 체크리스트 (요약)

- Env / Secret 마스킹 확인
- Port / Network 연결 OK
- 로그에 여러 없음
- 리소스 사용량 정상 범위
- Health / Readiness 정상
- Restart Policy 의도대로 동작

빠른 진단 명령 요약

- 로그: `docker logs -f <container>`
- 상태 정보: `docker inspect <container>`
- 내부 진입: `docker exec -it <container> sh`
- 리소스: `docker stats <container>`
- 재시작 정책: `docker inspect (RestartPolicy)`

예방 베스트 프랙티스

- 반드시 .env 버전 관리 (한정량 제외) + env-file 사용
- HEALTHCHECK 설정으로 조기 감지
- 명확한 이미지 태그 사용 (예: app:1.2.3)
- 네트워크/포트/볼륨 명명 규칙 일관성 유지
- 정기적인 정리 및 감사로 운영 비용 최적화

핵심 한 줄 요약
관찰하고, 확인하고, 원인을 찾아 해결한 뒤, 깨끗하게 정리하자!

수업 목표

- runtime config가 image build와 다른 단계임을 설명한다.
- e와 --env-file로 env를 주입하고 출력으로 확인한다.
- env 값이 image 안에 굳어진 값인지, 실행 시점에 들어간 값인지 구분한다.

개념 설명

Day 3에서 image를 만들었다면 Day 4는 그 image를 어떤 조건으로 실행할지 다룬다. 같은 image라도 APP_ENV=dev로 실행할 때와 APP_ENV=prod로 실행할 때 의미가 달라진다. 이 차이를 image를 다시 build해서 만들면 환경마다 image가 늘어나고, secret이 image layer에 남을 위험도 커진다.

Runtime config는 container를 시작할 때 들어가는 실행 조건이다. 대표적으로 env, port, volume, network, restart policy가 있다. 이 교시는 env부터 시작한다. 핵심은 값이 들어갔다가 아니라 어떤 방식으로 들어갔고, 어디에 남았고, 어떻게 확인했는가다.

현업 시나리오로 보면 더 분명하다. 같은 웹 애플리케이션 image를 개발 환경에서는 APP_ENV=dev, staging에서는 APP_ENV=staging, 운영에서는 APP_ENV=prod로 실행할 수 있다. 이때 환경마다 image를 다시 만들면 dev image, staging image, prod image가 서로 달라지고, 어떤 image가 실제 코드 변경 때문인지 설정 차이 때문인지 추적하기 어려워진다. 그래서 image는 가능한 한 동일하게 두고, 실행 조건을 runtime config로 바꾸는 방향을 기본값으로 둔다.

학생이 이 교시에서 잡아야 할 기준은 다음 한 문장이다.

image는 실행할 프로그램을 담고, runtime config는 그 프로그램이 어떤 환경으로 실행될지 정한다.

이 기준이 잡히면 Day 4의 나머지 내용도 자연스럽게 이어진다. logs는 실행 결과를 보는 도구이고, inspect는 어떤 runtime config가 적용됐는지 보는 도구이고, exec는 container 안에서 실제 상태를 확인하는 도구다.

환경설정을 파일로 로드할 때는 `docker run --env-file ./path/to/.env ...`를 쓴다. Docker는 이 파일을 container 생성 시점에 읽어서 container 환경변수로 넣는다. 파일을 나중에 수정해도 이미 만들어진 container에는 자동 반영되지 않는다. 값을 바꿨다면 container를 다시 생성해야 한다.

env file의 기본 형식은 다음과 같다.

형식	의미
APP_ENV=practice	값을 직접 지정
FEATURE_FLAG=on	값을 직접 지정
LOCAL_ONLY_KEY	host shell에 export된 값을 가져옴
# comment	comment로 무시

-e와 --env-file의 차이는 사용 목적이다.

방식	예시	적합한 상황	주의할 점
-e KEY=value	-e APP_ENV=practice	한두 개 값을 빠르게 바꿔 실험	명령줄과 shell history에 값이 남기 쉬움
--env-file FILE	--env-file .env	여러 설정을 파일로 묶어 재사용	파일을 명시하지 않으면 docker run이 자동으로 읽지 않음
-e KEY	-e APP_ENV	host shell의 export 값을 container로 전달	host에 값이 없으면 container에도 기대한 값이 없음

.env는 다음처럼 한 줄에 하나씩 기록한다.

```
APP_ENV=practice
FEATURE_FLAG=on
DB_PASSWORD=change-me-locally
# DEBUG=true
```

이 파일을 --env-file로 넘기면 container 안에서는 환경변수로 보인다. 파일 자체가 container 안에 복사되는 것이 아니라, 파일의 key/value가 container 환경으로 주입된다.

환경별로 파일을 나누는 것도 가능하다. 예를 들어 같은 image를 개발, staging, 운영 조건으로 나누고 싶다면 다음처럼 파일을 둘 수 있다.

```
.env.dev
.env.staging
.env.prod
```

각 파일은 같은 key를 가지되 값만 다르게 둔다.

```
# .env.dev
APP_ENV=dev
FEATURE_FLAG=on
API_BASE_URL=http://localhost:3000
```

```
# .env.staging
APP_ENV=staging
FEATURE_FLAG=on
API_BASE_URL=https://staging.example.com
```

```
# .env.prod
APP_ENV=prod
FEATURE_FLAG=off
API_BASE_URL=https://api.example.com
```

실행할 때는 어떤 환경 파일을 쓸지 명시한다.

```
docker run --rm --env-file .env.dev alpine:3.20 env
docker run --rm --env-file .env.staging alpine:3.20 env
docker run --rm --env-file .env.prod alpine:3.20 env
```

이 패턴은 Docker에서 끝나지 않는다. Kubernetes에서는 ConfigMap/Secret으로 환경별 설정을 분리하고, Terraform에서는 dev.tfvars, staging.tfvars, prod.tfvars처럼 환경별 variable 파일로 같은 생각을 확장한다. 지금 배우는 핵심은 특정 파일 이름이 아니라 **코드와 image는 최대한 동일하게 두고, 환경 차이는 설정으로 분리한다**는 원칙이다.

실습 명령

```
cd /mnt/d/paperclip
docker run --rm -e APP_ENV=practice -e FEATURE_FLAG=on alpine:3.20 env | sort
| grep -E 'APP_ENV|FEATURE_FLAG'
```

Expected:

```
APP_ENV=practice
FEATURE_FLAG=on
```

같은 image를 다른 env로 실행해 차이를 확인한다.

```
docker run --rm -e APP_ENV=dev alpine:3.20 sh -c 'echo "image=alpine:3.20
APP_ENV=$APP_ENV" '
docker run --rm -e APP_ENV=prod alpine:3.20 sh -c 'echo "image=alpine:3.20
APP_ENV=$APP_ENV" '
```

Expected:

```
image=alpine:3.20 APP_ENV=dev
image=alpine:3.20 APP_ENV=prod
```

해석: image는 둘 다 alpine:3.20으로 같지만 runtime config만 다르다. 이 차이를 이해해야 이후 nginx, PostgreSQL, Compose, Kubernetes에서도 config와 artifact를 섞지 않는다.

env file 확인

```
sed -n '1,120p' week2/day4/labs/env-report/.env.example
cp week2/day4/labs/env-report/.env.example week2/day4/labs/env-report/.env
docker run --rm --env-file week2/day4/labs/env-report/.env alpine:3.20 env |
sort | grep -E 'APP_ENV|FEATURE_FLAG'
```

Expected:

```
APP_ENV=practice
FEATURE_FLAG=on
```

.env 값이 container 안에서 어떻게 보이는지 확인한다.

```
docker run --rm --env-file week2/day4/labs/env-report/.env alpine:3.20 sh -c
'echo "APP_ENV=$APP_ENV FEATURE_FLAG=$FEATURE_FLAG DB_PASSWORD=$DB_PASSWORD" '
```

Expected:

```
APP_ENV=practice FEATURE_FLAG=on DB_PASSWORD=change-me-locally
```

문서나 screenshot에는 위처럼 실제 password 값을 남기지 않는다. 확인 후 기록할 때는 DB_PASSWORD=***masked***처럼 마스킹한다.

판단 기준

질문	확인 방법	좋은 답
env가 image build 없이 바뀌는가	같은 image를 다른 -e로 실행	실행 시점 값이 달라짐
-e와 --env-file 중 무엇을 쓰는가	값 개수와 재사용 여부 확인	임시 1개는 -e, 여러 설정은 env file
.env는 어떻게 쓰이는가	--env-file .env로 container 실행	파일 내용이 env로 주입됨
환경별 설정을 분리할 수 있는가	.env.dev, .env.staging, .env.prod 비교	같은 key, 다른 value로 환경 차이를 표현
env 값이 어디에 남는가	shell history, README, screenshot 확인	실제 secret 값은 남기지 않음
env file은 공유 가능한가	.env.example과 .env 구분	example만 공유, 실제 값은 local
env file 수정이 언제 반영되는가	container 재생성 후 확인	기존 container에는 자동 반영되지 않음
image와 runtime config를 구분하는가	같은 image를 다른 env로 실행	image는 같고 실행 조건만 달라짐

다음 연결

다음 교시는 .env.example과 실제 .env를 구분하고, secret을 노출하지 않는 기준을 다룬다.